

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN MÔN HỌC
MÔN: CƠ SỞ LẬP TRÌNH

HALO
CYBER ACCESS ENGINE

Hệ thống phân tích nhật ký truy cập tốc độ cao

Sinh viên thực hiện:

Bùi Văn Thiên

MSSV: 24120138

Lớp: 24CTT2

Giảng viên hướng dẫn:

Nguyễn Trọng Việt

Thành phố Hồ Chí Minh, tháng 5 năm 2026

Mục lục

1	Tổng Quan Đồ Án	3
1.1	Bối Cảnh	3
1.2	Yêu Cầu Chức Năng	3
1.3	Cấu Trúc Dữ Liệu Đầu Vào	3
2	Kiến Trúc Hệ Thống	5
2.1	Mô Hình Tổng Quan	5
2.2	Các Tầng Kiến Trúc	5
2.2.1	Tầng Giao Diện (Frontend)	5
2.2.2	Tầng Nghiệp Vụ & API (Backend)	6
2.2.3	Tầng Dữ Liệu	6
2.3	Đặc Tả API Endpoint	6
2.4	Luồng Xử Lý Một Truy Vấn	6
3	Tối Ưu Hóa Bộ Nhớ — Compact LogEntry	8
3.1	Vấn Đề về Quản Lý Bộ Nhớ Trong (In-Memory Processing)	8
3.2	Đặc Tả Kiến Trúc Cấu Trúc LogEntry 16-Byte	8
3.3	Hàm Chuyển Đổi ID	9
3.4	Hiệu Quả Sử Dụng Tài Nguyên Hệ Thống	10
4	Cấu Trúc Dữ Liệu Nền Tầng	11
4.1	Kiến Trúc Memory Pool Phân Khối	11
4.1.1	Vấn Đề Dữ Liệu	11
4.1.2	Cấu Trúc LogChunk	11
4.1.3	Trình Quản Lý Bể Nhớ LogPool	11
4.2	Danh Sách Động LogList	12
5	Hệ Thống Lập Chỉ Mục — Bảng Băm	13
5.1	Kiến Trúc Tổng Quan	13
5.2	Hàm Băm FNV-1a	13
5.3	Giải Quyết Xung Đột Bảng Dò Tuyến Tính	14
5.4	Tái Cấu Trúc Động — Rehashing	14
5.5	Tra Cứu Khóa Chuẩn Hóa	14
6	Thuật Toán Sắp Xếp và Tìm Kiếm	15

6.1	Sắp Xếp Với Cờ Trạng Thái	15
6.2	Giải Thuật Sắp Xếp Nhanh Không Độ Quy (Iterative QuickSort với Median-of-3)	15
6.3	Tìm Kiếm Nhị Phân Theo Mốc Thời Gian	17
6.4	Thuật Toán Top-10 Chèn Trực Tiếp	17
7	Nạp Dữ Liệu và Kiểm Soát Lỗi	19
7.1	Tách Trường CSV Nhanh Tránh Lỗi Biên	19
7.2	Xử Lý Lỗi Ba Lớp Tại API Server	19
8	Tầng Giao Diện Streamlit	21
8.1	Kiến Trúc Dashboard	21
8.2	Cải Tiến Trải Nghiệm Người Dùng	21
9	Hướng Dẫn Cài Đặt và Sử Dụng Hệ Thống	22
9.1	Yêu Cầu Môi Trường	22
9.2	Cấu Trúc Thư Mục Cần Thiết	22
9.3	Quy Trình Biên Dịch và Khởi Chạy	22
9.3.1	Bước 1: Biên dịch Backend C++	22
9.3.2	Bước 2: Khởi động API Server	23
9.3.3	Bước 3: Khởi động Giao diện Web (Frontend)	23
9.4	Thao Tác Trên Giao Diện (User Manual)	23
10	Tổng Kết và Đánh Giá	24
10.1	Đánh Giá Các Yêu Cầu Đề Bài	24
10.2	Đặc Tả Thiết Kế Kỹ Thuật Tối Ưu	24
10.3	Phân Tích Độ Phức Tạp	25
10.4	Định Hướng Phát Triển Cuối Kỳ	25

1. Tổng Quan Đề Án

1.1 Bối Cảnh

Một công ty công nghệ lớn gặp sự cố an ninh mạng nghiêm trọng. Hệ thống ghi nhận hàng triệu sự kiện truy cập từ người dùng, thiết bị và ứng dụng khác nhau. Đội ngũ an ninh không thể nhanh chóng phân tích dữ liệu vì lượng nhật ký (*log*) quá lớn và việc tìm kiếm diễn ra quá chậm chạp. Công ty quyết định xây dựng **Halo** — một *engine* phân tích truy cập giúp tổ chức dữ liệu hiệu quả, tìm kiếm nhanh và hỗ trợ phát hiện hành vi đáng ngờ.

1.2 Yêu Cầu Chức Năng

Chương trình C++ cần đáp ứng ba nhóm yêu cầu sau:

1. **Tìm kiếm nhanh trên dữ liệu lớn:** Hoạt động ổn định với tập dữ liệu tối thiểu 1.500.000 dòng, trả kết quả dưới 10 giây.
2. **Phân tích hoạt động truy cập:**
 - Truy vết hành trình của một người dùng (Device → App → Resource) trong khoảng thời gian cho trước.
 - Liệt kê toàn bộ lượt truy cập vào một tài nguyên (User → Device → App) trong khoảng thời gian cho trước.
 - Thống kê Top 10 tài nguyên được truy cập nhiều nhất.
3. **Phát hiện hành vi bất thường:** Chuẩn bị nền tảng cho giai đoạn cuối kỳ với các phát hiện dựa trên ngưỡng, hành vi, và phiên làm việc.

1.3 Cấu Trúc Dữ Liệu Đầu Vào

Dữ liệu được cung cấp dưới dạng file CSV. Dòng đầu là tiêu đề, các dòng tiếp theo là bản ghi thực tế.

Ví dụ hai dòng dữ liệu điển hình:

```
user_id , device_id , app_id , resource_id , event_type , location , timestamp
U03649 , D00205 , APP0380 , R02254 , ACCESS , KR , 1717682115
U03359 , D04468 , APP0045 , R04838 , DOWNLOAD , US , 1713999828
```

Bảng 1.1: Cấu trúc các cột trong file CSV đầu vào

STT	Tên cột	Diễn giải
1	user_id	Định danh người dùng (dạng U#####)
2	device_id	Định danh thiết bị người dùng đang sử dụng (D#####)
3	app_id	Định danh ứng dụng đang hoạt động (APP#####)
4	resource_id	Định danh tài nguyên đang được truy cập (R#####)
5	event_type	Loại sự kiện: LOGIN, LOGOUT, TOKEN_REFRESH, ACCESS, FAILED_LOGIN, OPEN_APP, DOWNLOAD, ADMIN_ACTION
6	location	Vị trí địa lý: US, VN, JP, KR, SG, CN, DE, FR, UK, AU, CA, IN, BR, RU, TH
7	timestamp	Thời điểm sự kiện xảy ra (Unix Epoch time, số nguyên)

2. Kiến Trúc Hệ Thống

2.1 Mô Hình Tổng Quan

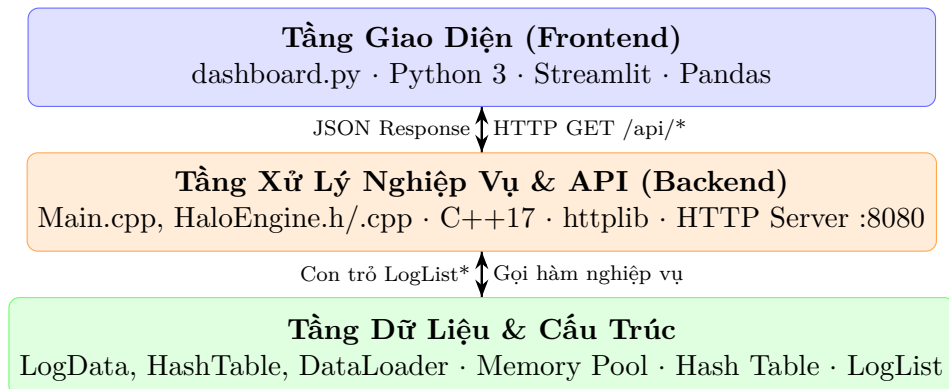
Hệ thống HALO được thiết kế theo mô hình **Client-Server phi trạng thái** (Stateless Client-Server Architecture). Thành phần xử lý dữ liệu và thành phần hiển thị giao diện được tách biệt hoàn toàn, giao tiếp qua giao thức **HTTP RESTful**.

Sự phân tách này mang lại hai lợi ích :

- **Tối ưu hiệu năng tính toán:** Toàn bộ cấu trúc dữ liệu, thuật toán tìm kiếm, băm và quản lý bộ nhớ phân khối được đặt ở Backend C++ để tận dụng tốc độ phần cứng tối đa.
- **Linh hoạt trong phát triển giao diện:** Frontend Python cho phép thay đổi trực quan hóa, bổ sung tính năng hiển thị mà không cần biên dịch lại lõi tính toán.

2.2 Các Tầng Kiến Trúc

Hệ thống phân thành ba tầng rõ ràng được mô tả trong Hình 2.1.



Hình 2.1: Sơ đồ kiến trúc ba tầng của hệ thống HALO

2.2.1 Tầng Giao Diện (Frontend)

- **Thành phần:** dashboard.py
- **Công nghệ:** Python 3.9+, Streamlit, Pandas

- **Chức năng:** Xây dựng giao diện của hệ thống, bao gồm bộ lọc thời gian, menu điều hướng, bảng dữ liệu và biểu đồ trực quan.
- **Giao tiếp:** Thực hiện HTTP GET tới Backend, nhận chuỗi JSON và chuyển đổi thành `pd.DataFrame` để hiển thị.

2.2.2 Tầng Nghiệp Vụ & API (Backend)

- **Thành phần:** `Main.cpp`, `HaloEngine.h/.cpp`
- **Công nghệ:** C++17, thư viện header-only `httplib.h`
- **Chức năng:** Khởi tạo HTTP Server lắng nghe tại cổng 8080; tiếp nhận request, điều phối đến `HaloEngine`, tuần tự hóa kết quả thành JSON bằng `std::ostringstream`.

2.2.3 Tầng Dữ Liệu

- **Thành phần:** `LogData.h/.cpp`, `HashTable.h/.cpp`, `DataLoader.h/.cpp`
- **Chức năng:** Đọc CSV, cấp phát Memory Pool, xây dựng chỉ mục Hash Table với độ phức tạp truy vấn $\mathcal{O}(1)$ trung bình.

2.3 Đặc Tả API Endpoint

Bảng 2.1: Danh sách các cổng giao tiếp RESTful API

Endpoint	Method	Tham số	Ý nghĩa
<code>/api/top</code>	GET	start, end	Top 10 tài nguyên được truy cập nhiều nhất
<code>/api/user</code>	GET	id, start, end	Hành trình của người dùng (Device→App→Resource)
<code>/api/resource</code>	GET	id, start, end	Lịch sử truy cập tài nguyên (User→Device→App)

Tất cả endpoint đều trả về Content-Type: `application/json` và thiết lập header `Access-Control-Allow-Origin`: để vượt rào cản CORS giữa Streamlit (cổng 8501) và Server C++ (cổng 8080).

2.4 Luồng Xử Lý Một Truy Vấn

Lấy ví dụ API truy vết người dùng GET `/api/user?id=U03649&start=...`

1. **Kích hoạt:** Người dùng nhập User ID và khoảng thời gian trên giao diện Streamlit, nhấn nút *Track User*.

2. **Gửi yêu cầu:** Python thực hiện HTTP GET với tham số đã đóng gói.
3. **Tiếp nhận & xác thực:** Main.cpp dùng khối try-catch để trích xuất tham số; nếu rỗng hoặc không hợp lệ, trả về mảng [] ngay lập tức.
4. **Tra cứu bộ nhớ:** HaloEngine gọi `userTable.search(id)` — $\mathcal{O}(1)$ — trả về con trỏ `LogList*`.
5. **Sắp xếp:** `sort()` kiểm tra cờ trạng thái; nếu đã sắp xếp từ trước, bỏ qua hoàn toàn.
6. **Thu hẹp khoảng thời gian:** Hai lần tìm kiếm nhị phân — $\mathcal{O}(\log N)$ — xác định chỉ mục đầu/cuối.
7. **Tuần tự hóa:** Duyệt cửa sổ chỉ mục, ghi JSON vào `ostreamstream`, tái tạo chuỗi ID từ số nguyên compact.
8. **Phản hồi:** Server trả về chuỗi JSON với HTTP 200. Streamlit chuyển thành `DataFrame` và hiển thị.

3. Tối Ưu Hóa Bộ Nhớ — Compact LogEntry

3.1 Vấn Đề về Quản Lý Bộ Nhớ Trong (In-Memory Processing)

Khi xây dựng một hệ thống xử lý và phân tích nhật ký truy cập (Log Analytics Engine) hoạt động hoàn toàn trên bộ nhớ trong (In-Memory) với quy mô dữ liệu lớn, việc tối ưu hóa dung lượng lưu trữ (Memory Footprint) của từng bản ghi là yếu tố cốt lõi quyết định tính ổn định. Do tính chất dữ liệu khổng lồ, việc kiểm soát chặt chẽ chi phí overhead quản lý vùng nhớ và con trỏ là bắt buộc nhằm ngăn chặn hiện tượng cạn kiệt tài nguyên hệ thống. Vì vậy, hệ thống triển khai giải pháp thiết kế cấu trúc dữ liệu dạng nén ở mức bit, chuyển đổi toàn bộ thông tin định danh thành định dạng số nguyên nhỏ nhằm đưa kích thước của mỗi bản ghi về mức tối giản tuyệt đối.

3.2 Đặc Tả Kiến Trúc Cấu Trúc LogEntry 16-Byte

Cấu trúc LogEntry được cấu trúc nhằm khớp chính xác với nguyên lý căn lề bộ nhớ tự nhiên của các kiến trúc vi xử lý 64-bit hiện đại, đạt kích thước cố định là **16 bytes**:

```
1 struct LogEntry {
2     uint32_t    user_id;        // Mã số nguyên định danh (4 bytes)
3     uint16_t    device_id;      // Mã thiết bị (2 bytes)
4     uint16_t    app_id;         // Mã ứng dụng (2 bytes)
5     uint16_t    resource_id;    // Mã tài nguyên (2 bytes)
6     EventType   event_type;     // hành vi (1 byte)
7     Location    location;       // vị trí (1 byte)
8     uint32_t    timestamp;      // Unix Epoch (4 bytes)
9 };
10 // sizeof(LogEntry) = 16 bytes (natural alignment, no padding
    needed)
```

Listing 3.1: Cấu trúc LogEntry tối ưu trong LogData.h

Kiểu vị trí và hành vi người dùng cũng được khai báo với kiểu nền tường minh để đảm bảo kích thước đúng 1 byte:

```
1 enum EventType : uint8_t {
2     LOGIN, LOGOUT, TOKEN_REFRESH, ACCESS,
```

```

3     FAILED_LOGIN, OPEN_APP, DOWNLOAD, ADMIN_ACTION, UNKNOWN_EVENT
4 };
5
6 enum Location : uint8_t {
7     US, VN, JP, KR, SG, CN, DE, FR, UK, AU, CA, IN, BR, RU, TH,
8     UNKNOWN_LOC
9 };

```

Listing 3.2: Enum với kiểu uint8_t**giải thuật nén dữ liệu:**

1. **Số nguyên hóa định danh:** Hệ thống thực hiện bóc tách phần tiền tố ký tự của các chuỗi định danh tại thời điểm nạp dữ liệu (ví dụ: chuỗi ‘"U03649"‘ được trích xuất thành giá trị số nguyên ‘3649‘ lưu vào uint32_t). Cơ chế này loại bỏ hoàn toàn việc sử dụng các đối tượng chuỗi ký tự động trong cấu trúc lưu trữ tĩnh, triệt tiêu chi phí trên vùng nhớ Heap.
2. **Thu hẹp kích thước kiểu liệt kê (uint8_t):** Hai kiểu dữ liệu phân loại hành vi (EventType) và quốc gia (Location) được ép kiểu về dạng uint8_t. Kỹ thuật này thu gọn không gian lưu trữ của mỗi trường xuống 1 byte, vừa với miền giá trị.
3. **Tối ưu hóa đa tầng bộ đệm (Cache Locality):** Kích thước 16 bytes là một số nguyên lũy thừa của 2, khớp với cấu trúc truyền dữ liệu (Cache Line alignment). Khi hệ thống tiến hành duyệt mảng liên tục, có thể nạp trọn vẹn nhiều bản ghi LogEntry vào bộ đệm L1/L2 của CPU chỉ trong duy nhất một chu kỳ truy xuất bộ nhớ, giảm thiểu tối đa hiện tượng hụt bộ đệm.

3.3 Hàm Chuyển Đổi ID

Việc mã hóa xuôi ngược giữa chuỗi định danh đầu vào và dạng số nguyên nén được đảm nhiệm bởi các hàm:

```

1 // Tai tao chuoai tu so nguyen (dung khi xuat JSON)
2 string userIdToStr    (uint32_t id) { return "U"    + zeroPad(id, 5)
3     ; }
4 string deviceIdToStr  (uint16_t id) { return "D"    + zeroPad(id, 5)
5     ; }
6 string appIdToStr     (uint16_t id) { return "APP"  + zeroPad(id, 4)
7     ; }
8 string resourceIdToStr(uint16_t id) { return "R"    + zeroPad(id, 5)
9     ; }

```

```
7 // Parse khi nạp CSV (chạy một lần duy nhất lúc khởi động)
8 uint32_t parseUserId (const string& s) { // "U03649" -> 3649
9     return (uint32_t) strtoul(s.c_str() + 1, nullptr, 10);
10 }
11 // Tương tự cho các Id khác...
```

Listing 3.3: Hàm parse và tái tạo chuỗi ID trong LogData.cpp

Các chuỗi tạm thời trong quá trình parse được cấp phát trên Stack và giải phóng ngay sau khi dòng CSV được xử lý xong.

3.4 Hiệu Quả Sử Dụng Tài Nguyên Hệ Thống

Dựa trên thiết kế cấu trúc 16 bytes cố định, tổng dung lượng RAM cần thiết để lưu giữ toàn bộ tập dữ liệu quy mô lớn (ví dụ 150 triệu dòng) được tính toán theo công thức:

$$\text{Tổng dung lượng RAM} = 150.000.000 \times 16 \text{ bytes} \approx 2,4 \text{ GB} \quad (3.1)$$

Mức tiêu thụ tài nguyên tối ưu này cho phép vận hành với hiệu năng cao trên các máy tính cá nhân thông thường, đồng thời để dành không gian bộ nhớ còn lại cho việc duy trì các cấu trúc chỉ mục động và bảng tra cứu nâng cao.

4. Cấu Trúc Dữ Liệu Nền Tảng

4.1 Kiến Trúc Memory Pool Phân Khối

4.1.1 Vấn Đề Dữ Liệu

Việc cấp phát nhiều khối Heap riêng lẻ bằng toán tử new cho từng LogEntry gây ra **phân mảnh bộ nhớ** và **chi phí Heap** lên tới 16 byte mỗi lần cấp phát. Để giải quyết, hệ thống triển khai kiến trúc cấp phát phân khối kết hợp.

4.1.2 Cấu Trúc LogChunk

LogChunk là một khối bộ nhớ liên tục chứa `CHUNK_SIZE = 1.000.000` bản ghi. Các bản ghi trong cùng một khối nằm liền kề nhau trong RAM, tối ưu hiệu ứng **Cache Locality**: khi CPU tải một phần tử vào bộ đệm L1/L2, các phần tử lân cận được tự động nạp theo, giảm triệt để hiện tượng hụt bộ đệm.

```
1  const int CHUNK_SIZE = 1000000; // 1 triệu bản ghi/khối
2
3  struct LogChunk {
4      LogEntry* entries; // mảng dòng liên tục
5      int count; // số bản ghi hiện tại
6      LogChunk* next; // liên kết tới khối tiếp theo
7      LogChunk();
8      ~LogChunk();
9  };
```

Listing 4.1: Cấu trúc LogChunk trong LogData.h

4.1.3 Trình Quản Lý Bể Nhớ LogPool

LogPool duy trì hai con trỏ:

- head: điểm bắt đầu của chuỗi khối, dùng cho duyệt toàn bộ và xây dựng chỉ mục.
- tail: trỏ thẳng đến khối đang ghi đang dở, cho phép cấp phát $\mathcal{O}(1)$ liên tục mà không cần duyệt lại từ đầu.

```
1  LogEntry* LogPool::allocateLog() {
2      // Neu day khoi hien tai, tao khoi moi
```

```

3     if (tail->count == CHUNK_SIZE) {
4         tail->next = new LogChunk();
5         tail      = tail->next;
6     }
7     LogEntry* newLog = &tail->entries[tail->count];
8     tail->count++;
9     total_logs++;
10    return newLog; // con tro vao vung nho goc, khong sao chep
11 }

```

Listing 4.2: Hàm cấp phát tối ưu trong LogPool

Độ phức tạp: $\mathcal{O}(1)$ trong mọi trường hợp. Hàm trả về con trỏ trực tiếp vào vùng bộ nhớ tĩnh, DataLoader ghi dữ liệu vào địa chỉ đó mà không tạo bản sao.

4.2 Danh Sách Động LogList

LogList là một mảng động các con trỏ LogEntry*, đóng vai trò danh sách nhật ký cho mỗi thực thể (một User, một Resource, ...). Thay vì lưu bản sao LogEntry, chỉ lưu con trỏ chiếu vào LogChunk, do đó mỗi bản ghi có thể xuất hiện trong nhiều bảng băm mà không tốn thêm bộ nhớ.

```

1 struct LogList {
2     LogEntry** data;
3     int        capacity;
4     int        count;
5     bool       isSorted;
6
7     LogList(int cap = 10);
8     ~LogList();
9     void push_back(LogEntry* log); // dat isSorted = false
10    void sort();
11 private:
12    void quickSort(int low, int high);
13 };

```

Listing 4.3: Cấu trúc LogList với cờ trạng thái isSorted

Cơ chế giãn nở push_back nhân đôi dung lượng mỗi khi đầy, đảm bảo độ phức tạp cấp phát trung bình $\mathcal{O}(1)$ (*amortized*).

5. Hệ Thống Lập Chỉ Mục — Bảng Băm

5.1 Kiến Trúc Tổng Quan

Hệ thống xây dựng ba bảng băm độc lập: `userTable` và `resourceTable` và `deviceTable`. Mỗi bảng ánh xạ một chuỗi định danh đến con trỏ `LogList*` chứa toàn bộ bản ghi liên quan đến thực thể đó.

Lưu ý: Bảng `deviceTable` tạm thời được bỏ qua ở bài nộp giữa kỳ, sẽ được tích hợp khi phát triển module bất thường ở giai đoạn tiếp theo.

5.2 Hàm Băm FNV-1a

Hệ thống sử dụng thuật toán **Fowler–Noll–Vo 1a (FNV-1a)** để chuyển chuỗi khóa thành vị trí số nguyên:

```
1 unsigned int HashTable::hashFunction(const string& key) {
2     unsigned int hash = 2166136261u; // FNV Offset Basis
3     for (unsigned char c : key) {
4         hash ^= c; // XOR byte hiện tại
5         hash *= 16777619u; // FNV Prime = 2^24 + 2^8 + 0x93
6     }
7     return hash % (unsigned int)capacity;
8 }
```

Listing 5.1: Hàm băm FNV-1a trong `HashTable.cpp`

Tính chất của FNV-1a:

- **FNV Offset Basis (2166136261):** Khởi tạo khác không, bảo toàn cấu trúc bit ngay cả khi chuỗi đầu vào bắt đầu bằng các ký tự giống nhau.
- **FNV Prime (16777619):** Số nguyên tố đảm bảo phép nhân liên tục trải đều các bit ra toàn bộ không gian 32-bit, tránh vòng lặp tuần hoàn.
- **Phép XOR trước nhân:** Một sự thay đổi nhỏ ở đầu vào làm biến đổi hoàn toàn mã băm đầu ra, phân phối đều đặn trên bảng.

5.3 Giải Quyết Xung Đột Bằng Dò Tuyến Tính

Khi xung đột xảy ra, hệ thống dùng **Linear Probing**: tính tiến tuần tự đến ô liền kề cho đến khi tìm ô trống hoặc đúng khóa. Để phòng trường hợp bảng đầy hoàn toàn, vòng lặp lưu lại `startIndex` và dừng khi quay trở lại điểm xuất phát.

5.4 Tái Cấu Trúc Động — Rehashing

Khi hệ số $\alpha = \text{size}/\text{capacity} \geq 0,7$, hàm `rehash()` được kích hoạt:

1. Cấp phát bảng mới kích thước $2 \times \text{capacity}$.
2. Duyệt toàn bộ bảng cũ, tính lại vị trí theo bảng mới.
3. **Chuyển giao quyền sở hữu con trỏ**: chỉ gán `table[i].logs = old_table[i].logs` không sao chép dữ liệu.
4. Giải phóng mảng cũ bằng `delete[] old_table`.

Ngưỡng 70% là điểm cân bằng tối ưu giữa sử dụng bộ nhớ hiệu quả và duy trì độ phức tạp tra cứu gần $\mathcal{O}(1)$.

5.5 Tra Cứu Khóa Chuẩn Hóa

Hệ thống cài đặt hàm tìm kiếm tập trung `HashTable::search()` để xử lý toàn bộ logic Linear Probing thay vì lặp lại code ở các API truy xuất:

```
1 string HaloEngine::getUserJourneyJson(const string& user_id, ...) {  
2     LogList* logs = userTable.search(user_id);  
3     if (!logs) return "[]";  
4  
5     logs->sort();  
6     int first = findFirstLogAfter(logs, start_time);  
7     int last  = findLastLogBefore(logs, end_time);  
8     if (first == -1 || last == -1 || first > last) return "[]";  
9     // ... xuất JSON su dung ham tai tao chuoi ID  
10 }
```

Listing 5.2: Tìm kiếm chuẩn hóa trong `HaloEngine.cpp`

6. Thuật Toán Sắp Xếp và Tìm Kiếm

6.1 Sắp Xếp Với Cờ Trạng Thái

Để tối ưu hóa thời gian tính toán, hệ thống duy trì cờ trạng thái bool `isSorted` bên trong `LogList`. Cờ này được đặt lại về false mỗi khi có lời gọi `push_back`.

```
1 void LogList::push_back(LogEntry* log) {
2     // ... gian no mang neu can
3     data[count++] = log;
4     isSorted = false; // moi entry co the pha vo thu tu hien tai
5 }
6
7 void LogList::sort() {
8     if (isSorted || count <= 1) return;
9     quickSort(0, count - 1);
10    isSorted = true;
11 }
```

Listing 6.1: Cờ `isSorted` trong `HashTable.cpp`

Nhờ cơ chế sắp xếp, mỗi `LogList` chỉ được sắp xếp **đúng một lần** trong toàn bộ vòng đời của chương trình, tiết kiệm chu kỳ CPU cho các truy vấn lặp lại.

6.2 Giải Thuật Sắp Xếp Nhanh Không Độ Quy (Iterative QuickSort với Median-of-3)

Để đảm bảo an toàn về mặt bộ nhớ và loại bỏ rủi ro tràn ngăn xếp khi xử lý các tập log có số lượng phần tử lên tới hàng triệu bản ghi cho một thực thể, hệ thống triển khai giải thuật Sắp xếp nhanh dạng vòng lặp kết hợp kỹ thuật chọn phần tử chốt trung vị (Median-of-3).

```
1 void LogList::quickSort(int low, int high) {
2     // Khoi tao mang Stack thu cong tren vung nho Heap
3     int stackCap = (high - low + 1) * 2;
4     int* stack = new int[stackCap];
5     int top = -1;
6 }
```

```

7     stack[++top] = low;
8     stack[++top] = high;
9
10    while (top >= 0) {
11        int h = stack[top--];
12        int l = stack[top--];
13        if (l >= h) continue;
14
15        //Chon pivot theo ky thuat Median-of-Three
16        int m = l + (h - l) / 2;
17        if (data[l]->timestamp > data[m]->timestamp) { swap(data[l]
18], data[m]); }
19        if (data[l]->timestamp > data[h]->timestamp) { swap(data[l]
20], data[h]); }
21        if (data[m]->timestamp > data[h]->timestamp) { swap(data[m]
22], data[h]); }
23        // data[m] gio la trung vi, chuyen xuong cuoi de dung lam
24        pivot
25        { LogEntry* t = data[m]; data[m] = data[h]; data[h] = t; }
26
27        //Phan vung
28        uint32_t pivot = data[h]->timestamp;
29        int i = l - 1;
30        for (int j = l; j < h; j++) {
31            if (data[j]->timestamp <= pivot) {
32                i++;
33                LogEntry* t = data[i]; data[i] = data[j]; data[j] =
34                t;
35            }
36        }
37        { LogEntry* t = data[i+1]; data[i+1] = data[h]; data[h] = t
38; }
39        int pi = i + 1;
40
41        if (pi - 1 > l) { stack[++top] = l;        stack[++top] = pi
42- 1; }
43        if (pi + 1 < h) { stack[++top] = pi + 1; stack[++top] = h;
44        }
45    }
46    delete[] stack;
47}

```

Listing 6.2: QuickSort lặp với pivot trung vị ba trong HashTable.cpp**Đặc điểm thiết kế giải thuật:**

1. **Kiểm soát không gian bộ nhớ hệ thống:** Việc tự thiết lập một cấu trúc dữ liệu Stack nằm trên vùng nhớ Heap giúp giải thuật hoàn toàn độc lập và không phụ thuộc vào giới hạn dung lượng của Call Stack thuộc Hệ điều hành. Nhờ đó, hệ thống có thể xử lý các chuỗi log có độ sâu phân hoạch lớn mà vẫn đảm bảo tính an toàn vùng nhớ.
2. **Loại bỏ trường hợp xấu nhất:** Phép tính lấy trung vị giữa 3 phần tử (Đầu, Giữa, Cuối) làm phần tử chốt (Pivot) giúp giải thuật triệt tiêu rủi ro rơi vào độ phức tạp thời gian suy biến $\mathcal{O}(N^2)$ khi gặp phải các tập dữ liệu log đã gần như có thứ tự sẵn. Cơ chế này đảm bảo hiệu năng sắp xếp của hệ thống luôn duy trì ở mức lý tưởng.

6.3 Tìm Kiếm Nhị Phân Theo Mốc Thời Gian

Sau khi LogList được sắp xếp tăng dần theo timestamp, hai hàm tìm kiếm nhị phân chuyên biệt xác định cửa sổ chỉ mục trong $\mathcal{O}(\log N)$:

- `findFirstLogAfter(list, start_time)`: tìm chỉ mục nhỏ nhất i thỏa `list[i].timestamp  $\geq$  start_time`.
- `findLastLogBefore(list, end_time)`: tìm chỉ mục lớn nhất j thỏa `list[j].timestamp  $\leq$  end_time`.

Với User có 100.000 bản ghi, hệ thống cần tối đa $\lceil \log_2(100000) \rceil = 17$ phép so sánh để xác định chính xác điểm đầu và điểm cuối thay vì duyệt tuyến tính 100.000 phần tử.

6.4 Thuật Toán Top-10 Chèn Trực Tiếp

```

1 static void insertTop10(ResourceCount top10[10], uint16_t res_id,
   int cnt) {
2     if (cnt < top10[9].count) return;
3     if (cnt == top10[9].count && res_id >= top10[9].resource_id)
       return;
4
5     int j = 9;
6     while (j > 0 && (top10[j-1].count < cnt ||
7         (top10[j-1].count == cnt && top10[j-1].resource_id >
           res_id))) {

```

```
8         top10[j] = top10[j - 1];
9         j--;
10    }
11    top10[j].resource_id = res_id;
12    top10[j].count      = cnt;
13 }
```

Listing 6.3: Hàm chèn Top-10 tối ưu trong HaloEngine.cpp

Thay vì tích lũy tất cả rồi sắp xếp $\mathcal{O}(M \log M)$, hệ thống duy trì một mảng cố định 10 phần tử. Mỗi tài nguyên chỉ cần một lần so sánh với phần tử cuối bảng. Toàn bộ thuật toán đạt $\mathcal{O}(M)$ với M là số tài nguyên phân biệt, và tỷ lệ chèn vào top10 là rất nhỏ nên vòng while bên trong gần như chạy rất ít.

7. Nạp Dữ Liệu và Kiểm Soát Lỗi

7.1 Tách Trường CSV Nhanh Tránh Lỗi Biên

Hàm `splitLine()` được thiết kế để tách các trường CSV thông qua cấu trúc con trỏ hằng, không cấp phát chuỗi trung gian. Hệ thống đồng thời đếm chính xác số cột và từ chối xử lý đối với các dòng dữ liệu không hợp lệ:

```
1 static bool splitLine(const string& line ,
2                       const char* fields[EXPECTED_FIELDS] ,
3                       size_t      lengths[EXPECTED_FIELDS])
4 {
5     const char* p    = line.c_str();
6     const char* end = p + line.size();
7     int col = 0;
8     fields[0] = p;
9
10    while (p <= end && col < EXPECTED_FIELDS) {
11        if (p == end || *p == ',') {
12            lengths[col] = (size_t)(p - fields[col]);
13            col++;
14            if (col < EXPECTED_FIELDS) fields[col] = p + 1;
15        }
16        p++;
17    }
18    return col == EXPECTED_FIELDS; // true: đủ 7 cột; false: bị lỗi
19 }
```

Listing 7.1: Hàm tách trường an toàn trong `DataLoader.cpp`

Dòng không đủ 7 cột bị bỏ qua và hệ thống vẫn chạy tiếp tục, đảm bảo tính an toàn khi gặp rủi ro của Server.

7.2 Xử Lý Lỗi Ba Lớp Tại API Server

Tầng API trong `Main.cpp` được bảo vệ bởi ba lớp kiểm soát:

1. **Lớp 1 — Tham số rỗng:** Kiểm tra `id.empty()` trước khi gọi bất kỳ hàm nào, trả về `[]` ngay lập tức.

2. **Lớp 2 — Ngoại lệ toàn cục:** Toàn bộ khối xử lý được bọc trong try-catch. Bất kỳ lỗi nào từ HaloEngine cũng bị bắt và trả về HTTP 500 thay vì để Server sập.
3. **Lớp 3 — CORS:** Header Access-Control-Allow-Origin: được thiết lập cho mọi phản hồi, ngăn trình duyệt chặn request từ Streamlit chạy trên cổng khác.

8. Tầng Giao Diện Streamlit

8.1 Kiến Trúc Dashboard

Giao diện được xây dựng với nền tối và accent màu xanh dương, tổ chức thành bốn module điều hướng độc lập:

Bảng 8.1: Bốn module chức năng của Dashboard

Module	API gọi	Chức năng
Overview – Top 10	/api/top	Hiển thị bảng tần suất và biểu đồ cột Top 10 tài nguyên
User Journey	/api/user	Bảng hành trình, biểu đồ timeline theo giờ, thống kê thiết bị/ứng dụng
Resource Tracker	/api/resource	Bảng lịch sử truy cập, biểu đồ tần suất, top người dùng truy cập
Anomaly Radar	—	Danh sách module phát hiện bất thường, chuẩn bị cho cuối kỳ

8.2 Cải Tiến Trải Nghiệm Người Dùng

Giao diện ứng dụng cung cấp các thiết kế hiện đại:

- **Date picker tích hợp:** Người dùng chọn ngày trực quan, hệ thống tự động chuyển đổi sang Unix Epoch, loại bỏ sai sót khi nhập số.
- **Thanh metric tóm tắt:** Mỗi kết quả truy vấn hiển thị ngay các chỉ số tổng hợp (tổng sự kiện, số thiết bị, khoảng thời gian hoạt động).
- **Tab đa chiều:** Mỗi module tổ chức kết quả qua nhiều tab (Event Log, Timeline, Breakdown) thay vì một bảng phẳng duy nhất.
- **Quick Anomaly Hints:** Kiểm tra sơ bộ ngay trên Frontend (ví dụ: User dùng > 4 thiết bị) để định hướng điều tra mà không cần gọi thêm API.

9. Hướng Dẫn Cài Đặt và Sử Dụng Hệ Thống

9.1 Yêu Cầu Môi Trường

Hệ thống được thiết kế tối ưu với cấu hình và môi trường khuyến nghị như sau:

- **Trình biên dịch:** g++ hoặc clang++ hỗ trợ chuẩn C++17 trở lên.
- **Ngôn ngữ Frontend:** Python 3.9 trở lên.
- **Thư viện Python:** streamlit, pandas, requests.

9.2 Cấu Trúc Thư Mục Cần Thiết

Trước khi bắt đầu biên dịch, thư mục gốc của đồ án cần được tổ chức chính xác theo mô hình dưới đây:

```
HALO_CYBER_ACCESS_ENGINE/  
    data/  
        halo_dataset_1_5m.csv    # Tập dữ liệu log thực tế  
    src/  
        Main.cpp  
        HaloEngine.cpp  
        ... (các file mã nguồn .cpp, .h)  
    release/                        # Thư mục chứa file thực thi  
sau biên dịch  
        dashboard.py              # File giao diện Python  
    Streamlit
```

9.3 Quy Trình Biên Dịch và Khởi Chạy

9.3.1 Bước 1: Biên dịch Backend C++

Mở ứng dụng Terminal tại thư mục gốc của dự án và chạy lệnh biên dịch sử dụng cờ tối ưu hóa -O3 để đạt hiệu năng tối đa:

```
g++ -O3 -std=c++17 src/*.cpp -o release/halo_engine
```

Sau khi thực thi lệnh, nếu không có thông báo lỗi nào xuất hiện, quá trình đúc file thực thi đã hoàn tất thành công.

9.3.2 Bước 2: Khởi động API Server

Tiến hành khởi chạy tập tin thực thi vừa được tạo ra:

```
./release/halo_engine
```

Hệ thống sẽ tiến hành đọc tệp CSV và nạp dữ liệu vào Memory Pool. Khi cấu trúc chỉ mục hoàn tất, màn hình Console sẽ hiển thị thông báo: [NETWORK] API Server đang chạy tại: <http://localhost:8080>. Lúc này, Backend đã sẵn sàng phục vụ các truy vấn mạng.

9.3.3 Bước 3: Khởi động Giao diện Web (Frontend)

Mở một tab Terminal mới (lưu ý giữ nguyên tab Backend đang chạy), tiến hành cài đặt các thư viện cần thiết và khởi động Streamlit:

```
pip install streamlit pandas requests
streamlit run dashboard.py
```

9.4 Thao Tác Trên Giao Diện (User Manual)

Sau khi thực thi lệnh khởi chạy Frontend, trình duyệt web sẽ tự động mở địa chỉ <http://localhost:8501> với giao diện điều khiển trung tâm (Dashboard). Các bước thao tác chuẩn:

- Cấu hình Thời gian (Sidebar):** Tại thanh điều hướng bên trái, chọn khoảng ngày cần phân tích (ví dụ: từ 01/04/2024 đến 30/04/2024). Cấu hình này sẽ tự động đóng gói tham số Epoch Time để áp dụng xuyên suốt cho mọi truy vấn của các module.
- Xem Overview (Top 10 Resources):** Chọn tab *Overview*. Hệ thống tự động gửi yêu cầu lấy thông tin 10 tài nguyên được truy cập nhiều nhất trong khoảng thời gian đã định, xuất hiện biểu đồ tiến trình và KPI tóm tắt.
- Truy vết Người dùng (User Journey):** Chuyển sang tab *User Journey*, nhập mã định danh người dùng (ví dụ: U03649) và nhấn "Track User". Kết quả được chia thành ba góc nhìn chuyên sâu: Bảng Log thô (Dataframe), Biểu đồ Dòng thời gian (Timeline), và Thống kê tần suất.
- Kiểm tra Bất thường:** Tại mỗi kết quả trả về, module *Quick Anomaly Hints* sẽ tự động kích hoạt các luật kiểm tra sơ bộ và hiển thị thẻ cảnh báo nếu phát hiện người dùng sử dụng số lượng thiết bị bất thường.

10. Tổng Kết và Đánh Giá

10.1 Đánh Giá Các Yêu Cầu Đề Bài

Bảng 10.1: Kiểm tra đáp ứng yêu cầu đề bài giữa kỳ

Yêu cầu	Trạng thái	Ghi chú
Tự thiết kế cấu trúc dữ liệu	✓	LogPool, LogList, HashTable
Nạp dữ liệu CSV	✓	Có xác thực số cột
Lưu trữ và quản lý bộ nhớ	✓	Memory Pool phân khối
Sắp xếp, đánh chỉ mục	✓	Sort với isSorted
Hành trình người dùng	✓	getUserJourneyJson
Hành trình tài nguyên	✓	getResourceJourneyJson
Top 10 tài nguyên theo thời gian	✓	Thuật toán chèn $\mathcal{O}(M)$
Giải phóng bộ nhớ đầy đủ	✓	Destructor đầy đủ ở mọi tầng
Xử lý dữ liệu không hợp lệ	✓	splitLine() bỏ qua dòng lỗi
Hoạt động > 1.5 triệu dòng	✓	16 byte/bản ghi, 2.4 GB @ 150M

10.2 Đặc Tả Thiết Kế Kỹ Thuật Tối Ưu

Bảng 10.2: Đặc tả thông số thiết kế và Giải pháp Tối ưu hóa Hạ tầng

Thành phần	Giải pháp Hiện thực	Hiệu quả Kỹ thuật
LogEntry	Số nguyên hóa + Nén bit (uint8_t)	Kích thước cố định 16 bytes, tổng RAM cho 150 triệu dòng chiếm tối đa $\approx$ 2.4 GB.
Đọc File	Zero-Copy Parsing (splitLine())	Sử dụng mảng con trả hàng, không tạo chuỗi trung gian, ngăn chặn lỗi thiếu cột.
Vòng đời	Danh sách liên kết phân khối (LogChunk)	Loại bỏ phân mảnh bộ nhớ khi sử dụng lệnh new liên tục.
Chỉ mục	Bảng băm mở rộng động (rehash())	Duy trì độ phức tạp tra cứu $\mathcal{O}(1)$ ngay cả khi tải lượng dữ liệu khổng lồ.
Sắp xếp	Phi đệ quy + Median-of-3 + isSorted	Chống tràn Call Stack, chuyển độ phức tạp của các truy vấn trùng lặp về $\mathcal{O}(1)$.

10.3 Phân Tích Độ Phức Tạp

Bảng 10.3: Độ phức tạp thời gian và không gian của các thao tác chính

Thao tác	Thời gian	Không gian	Ghi chú
Nạp CSV	$\mathcal{O}(N)$	$\mathcal{O}(N)$	N = số dòng
Xây dựng chỉ mục	$\mathcal{O}(N)$	$\mathcal{O}(N)$	Insert mỗi bản ghi
Tra cứu Hash Table	$\mathcal{O}(1)^*$	—	* trung bình
Sort LogList	$\mathcal{O}(K \log K)$	$\mathcal{O}(K)$	K = số log của thực thể
Sort lần 2 trở đi	$\mathcal{O}(1)$	$\mathcal{O}(1)$	Nhờ cờ isSorted
Binary search	$\mathcal{O}(\log K)$	$\mathcal{O}(1)$	—
Top 10 Resources	$\mathcal{O}(M)$	$\mathcal{O}(1)$	M = số tài nguyên phân biệt

10.4 Định Hướng Phát Triển Cuối Kỳ

Dựa trên nền tảng hiện tại, các module phát hiện bất thường sau sẽ được triển khai:

- **Brute Force:** Phát hiện chuỗi FAILED_LOGIN liên tiếp vượt ngưỡng trong cửa sổ thời gian.
- **Multi-Device Anomaly:** Một người dùng đăng nhập từ $\geq N$ thiết bị phân biệt trong khoảng thời gian ngắn.
- **Geo-Hopping:** Phát hiện sự dịch chuyển địa lý không thể xảy ra giữa hai sự kiện liên tiếp.
- **Off-Hours Access:** Sự kiện ngoài giờ làm việc (00h–06h theo múi giờ cục bộ).
- **Silent-then-Burst:** Người dùng im lặng > 7 ngày rồi đột ngột hoạt động mạnh.
- **Session Hijacking Risk:** Phiên làm việc có chuỗi ADMIN_ACTION + DOWNLOAD liên tục bất thường.

Để hỗ trợ các module này, các cấu trúc đánh giá thời gian (Time-gap analysis) và phân tích phiên (*session*) sẽ được mở rộng kết hợp cùng HashTable.